

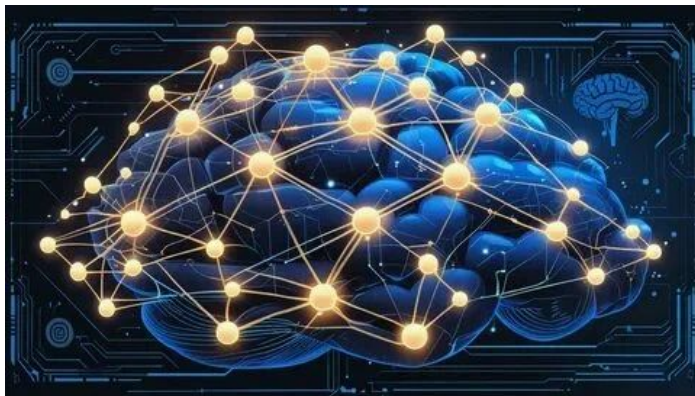
Нейронная сеть прямого распространения (классификатор)

Осипов Александр, 25222

Что такое нейронная сеть прямого распространения?

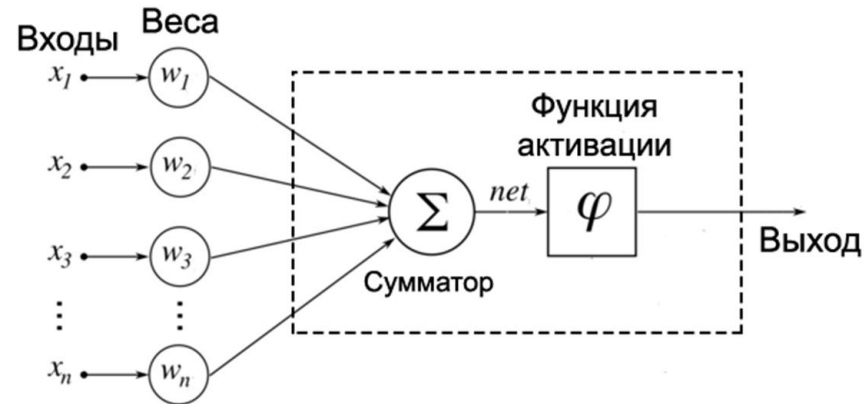
Нейронная сеть прямого распространения (англ. *feedforward neural network*) — это самый базовый тип искусственной нейронной сети, где информация движется **только в одном направлении**: от входа к выходу, без циклов и обратных связей.

В случае классификации, нейронная сеть предсказывает класс, к которому относится объект.



Архитектура нейронной сети. Нейрон

Нейрон — это базовый элемент искусственной нейронной сети, своего рода «мини-вычислитель», который принимает входные данные, обрабатывает их и передаёт дальше.



Итого на выходе нейрона получаем $\varphi(w_1 \cdot x_1 + \dots + w_n \cdot x_n)$



Архитектура нейронной сети. Функция активации

Функция активации — это то, что делает нейрон нелинейным и решает, какой сигнал он передаст дальше. Без неё нейронная сеть была бы просто линейной комбинацией чисел — и не могла бы учить сложные зависимости.

Зачем она нужна?

После того как нейрон посчитал взвешенную сумму входов, функция активации:

- + «решает», насколько сильно нейрон активируется
- + добавляет нелинейность ограничивает
- + преобразует значение

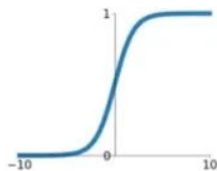
Без активаций даже глубокая сеть свелась бы к одной линейной функции — почти бесполезной для реальных задач.



Архитектура нейронной сети. Функция активации

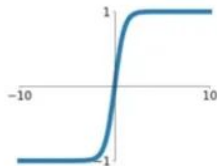
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



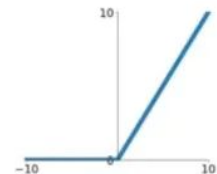
tanh

$$\tanh(x)$$



ReLU

$$\max(0, x)$$



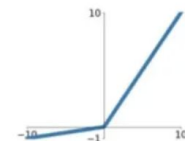
ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Leaky ReLU

$$\max(0.1x, x)$$

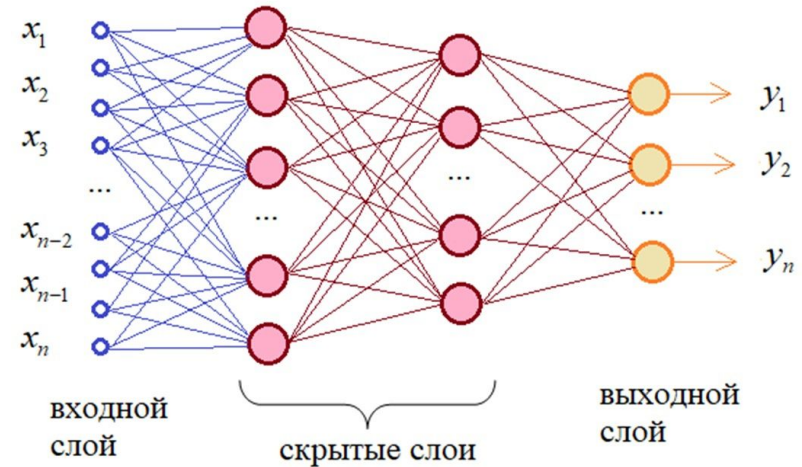


Архитектура нейронной сети. Слои

Входной слой - это первый слой, который получает данные. Он не обучается, просто передает данные дальше

Скрытые слои - это ядро сети, здесь происходит обучение

Выходной слой - это последний слой, который выдает результат



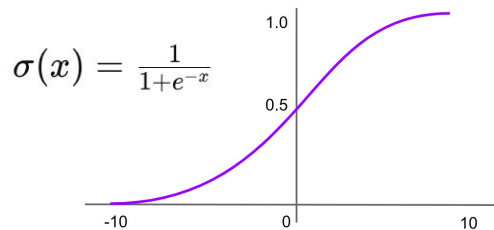


Архитектура нейронной сети.

Выходной слой

1. Бинарная классификация

Если нужно выбрать между двумя вариантами, обычно используется **один нейрон с сигмодой**.



2. Многоклассовая классификация

Если классов несколько, используется **softmax-слой**, он превращает набор чисел в вероятности, которые суммируются в 1.

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Пример: кошка=0.6, собака = 0.3, заяц = 0.1



Функция потерь

Функция потерь (loss function) — это способ измерить, насколько сильно предсказание нейронной сети отличается от правильного ответа.

Для классификации используется **Cross-Entropy**

$$L = - \sum_i y_i \log(p_i)$$

где i - i -й класс, y_i - 0 или 1 (является ли объект i -м классом), p_i - вероятность



Обучение

Обучение модели = минимизация функции потерь

Общая идея обучения

1. Нейросеть получает вход
2. делает предсказание
3. сравнивает с правильным ответом (loss)
4. исправляет свои веса



Backpropagation. Введение

Функцию потерь можно представить как $L=f(x;W)$. Наша задача - подобрать такие W , чтобы L была минимальной. Решать эту задачу напрямую алгебраически не вариант.

Для этого нужно знать, как каждый вес w влияет на ошибку:

$$\frac{\partial L}{\partial W}$$

Проблема: веса влияют на ошибку не напрямую, поскольку L является на самом деле сложной функцией.

$$x \rightarrow W_1 \rightarrow \text{ReLU} \rightarrow W_2 \rightarrow \text{ReLU} \rightarrow W_3 \rightarrow \text{softmax} \rightarrow \text{loss}$$

Чтобы посчитать $\frac{\partial L}{\partial W}$ используют **chain rule**.



Backpropagation. Chain rule

Если

$$L = L(a), \quad a = f(b), \quad b = g(w)$$

то:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial b} \cdot \frac{\partial b}{\partial w}$$

Можно сказать, что chain rule превращает «сложную зависимость» в «произведение простых локальных зависимостей»



Backpropagation. Обновление весов (градиентный спуск)

Backpropagation работает следующим образом:

1. Считаем производную loss по выходу
2. Идём назад по слоям
3. Считаем, как каждый вес повлиял на ошибку (chain rule)
4. **Обновляем веса** по формуле

$$w := w - \eta \frac{\partial L}{\partial w}$$

где:

- w — вес
- η — скорость обучения (learning rate)
- $\frac{\partial L}{\partial w}$ — насколько вес влияет на ошибку



Batch

Batch (батч) — это небольшая группа обучающих примеров, которую нейросеть обрабатывает за один шаг обучения.

Вместо того чтобы обновлять веса после каждого одного примера или считать градиент по всему датасету, обычно берут небольшой набор примеров одновременно.

SGD

$$L_{batch} = \frac{1}{N} \sum_{i=1}^N L_i$$

batch size = 1



Mini-batch (стандарт)

32 / 64 / 128 / 256



Full batch

весь датасет сразу

